

# Concurrency on Modern Multi-Core CPUs

Instruction Reordering • Memory Barriers • MESI/MOESI • False Sharing • CAS • Lock-Free Ring Buffers

## EPAM CoE Research Assignment

A hardware-level explanation of why concurrent programs can fail—and how high-performance lock-free systems avoid unnecessary synchronization costs.

## Executive Summary

Modern multi-threaded software cannot assume that a write performed by one thread becomes immediately visible to every other thread in exactly the same order. Compilers may reorder independent operations, and modern CPUs execute memory operations using out-of-order pipelines, store buffers, speculative loads, and private caches. The uploaded research document identifies this as a central reason that apparently correct concurrent programs can fail on some executions or architectures. ■filecite■turn0file0■L4-L16■

Correctness therefore requires an explicit memory model. Acquire/release operations establish a **happens-before** relationship between a publisher and a consumer, while sequential consistency provides a stronger global ordering. Separately, cache coherence protocols such as MESI/MOESI keep caches coherent, but coherence alone does not solve memory-ordering problems. ■filecite■turn0file0■L69-L95■

Performance introduces another layer: CPUs transfer and invalidate data at **cache-line granularity**, commonly 64 bytes. Two independent variables can therefore interfere if they occupy the same cache line. This is called **false sharing**. Lock-free structures such as ring buffers and the LMAX Disruptor address these costs through atomic operations, careful memory ordering, cache-line padding, and preallocated storage.

■filecite■turn0file0■L114-L165■

## 1. Why Does Instruction Reordering Happen?

At first glance, a programmer may think instructions execute exactly in source-code order. Modern systems deliberately break this assumption when doing so improves performance while preserving the behavior expected by a single-threaded program.

**There are two important sources of reordering:**

- **Compiler reordering:** The compiler can rearrange independent instructions during optimization as long as the resulting single-threaded behavior remains valid.
- **CPU/hardware reordering:** A CPU can execute instructions out of order. Stores may wait in a store buffer while later instructions continue executing, and loads may be speculatively executed.

The research document gives the classic message-passing example: Thread 1 writes data = 42 and then sets ready = true; Thread 2 waits for ready and then reads data. The programmer expects Thread 2 to see 42. Without appropriate synchronization, that assumption is not guaranteed on weaker memory architectures.

■filecite■turn0file0■L20-L42■

### 1.1 The Race in Simple Terms

```
Thread 1 Thread 2 ----- data = 42; while (!ready) {} ready = true; print(data);
```

The intended order is: **write data** → **publish ready** → **read ready** → **read data**. The problem is that visibility between cores is not instantaneous. Thread 1 can have the data write sitting in a store buffer while the publication of the flag becomes observable. Thread 2 can therefore observe the flag and still lack the correctly ordered visibility of the data.

**Important distinction:** this is not simply “the CPU executed the wrong instruction.” The language/compiler memory model and the hardware memory model determine which observations are legal. A correct concurrent program must explicitly establish the ordering it requires.

**Why can unit tests miss it?** The problematic interleaving is timing-dependent and may be rare. On x86, the stronger x86-TSO memory model makes many reorderings less visible than on weaker architectures such as ARM. The uploaded research specifically notes that a test may appear reliable on x86 and still expose a race on weaker-memory hardware. ■filecite■turn0file0■L49-L65■

## 2. Memory Barriers: What Do They Actually Do?

A memory barrier, or fence, is a synchronization mechanism that restricts the order in which memory operations may become visible. A useful mental model is: “**Do not allow memory operations on the wrong side of this boundary to pass each other.**”

At the microarchitectural level, a fence can constrain the CPU's store buffers, speculative execution, and retirement of memory operations. The uploaded research describes a full x86 MFENCE as waiting for the store buffer to drain before later memory operations proceed. ARM uses mechanisms such as DMB, and acquire/release instructions such as LDAR and STLR. ■filecite■turn0file0■L69-L99■

**Important nuance:** acquire/release operations should not be thought of as “flush every cache immediately.” Their primary purpose is **ordering**. The hardware implements the required ordering using architecture-specific mechanisms; the exact microarchitectural behavior differs between CPUs.

### 2.1 Acquire and Release

**Release = publish my previous work.**

A release store prevents earlier memory operations from being reordered after the publication point.

**Acquire = safely consume published work.**

An acquire load prevents later memory operations from being reordered before the acquire point.

```
Thread 1 (Producer) Thread 2 (Consumer) data = 42; while (!ready.load(acquire)) {} ready.store(true,
release); print(data);
```

If Thread 2's acquire load observes the value written by Thread 1's release store, the release/acquire pair establishes a **happens-before** relationship. The producer's earlier write to data is therefore ordered before the consumer's read. ■filecite■turn0file0■L69-L84■

### 2.2 Sequential Consistency

Sequential consistency (SC) is stronger. Conceptually, all sequentially consistent atomic operations appear in one global order that is consistent with each thread's program order. C++ exposes this as `memory_order_seq_cst`. The stronger guarantee can require additional ordering machinery and can therefore be more expensive than weaker ordering when hardware needs extra barriers. ■filecite■turn0file0■L85-L95■

| Memory order    | Simple interpretation                                       | Typical use                                                     |
|-----------------|-------------------------------------------------------------|-----------------------------------------------------------------|
| Relaxed         | Atomic operation, but no synchronization ordering.          | Counters/statistics where ordering is irrelevant.               |
| Acquire         | Do not allow later reads/writes to move before this point.  | Consumer side of a publication.                                 |
| Release         | Do not allow earlier reads/writes to move after this point. | Producer side of a publication.                                 |
| Acquire-Release | Combines acquire and release semantics.                     | Read-modify-write operations such as some CAS/fetch operations. |

| Memory order            | Simple interpretation                    | Typical use                                                                     |
|-------------------------|------------------------------------------|---------------------------------------------------------------------------------|
| Sequentially consistent | Strong global ordering of SC operations. | When a simple global ordering is more important than minimizing ordering costs. |

The uploaded research summarizes these same C++ memory orders and their typical uses.

filecite turn0file0L523-L566

### 3. Memory Race: State Transition / Happens-Before Diagram

The following model shows the intended synchronization path.

```

PRODUCER CORE CONSUMER CORE ----- 1. data = 42 | v 2. ready.store(true,
RELEASE) | | publication / synchronization |-----> 3. ready.load(ACQUIRE)
| v 4. read data => data == 42 Without release/acquire: data write and ready publication do not have
the required cross-thread ordering. With release/acquire: data write ---> release ---> acquire --->
data read happens-before relationship

```

**Key idea:** the fence is not primarily about making the program “wait.” It establishes an ordering rule that allows the consumer to safely interpret the published state.

## 4. MESI/MOESI: How Cache Coherence Works

Every CPU core has private caches. If two cores cache the same memory line, the system needs a protocol to keep their copies coherent. MESI is one such protocol.

| State         | Meaning                                                                      |
|---------------|------------------------------------------------------------------------------|
| M — Modified  | This cache has the newest/dirty copy; other caches do not have a valid copy. |
| E — Exclusive | This cache owns the clean line and no other core is caching it.              |
| S — Shared    | Multiple cores may have a clean copy of the line.                            |
| I — Invalid   | This cache's copy cannot be used.                                            |

These four states and their meanings are described in the uploaded research. ■filecite■turn0file0■L114-L131■

### 4.1 Example State Transition

Initial: Core 0: X = S Core 1: X = S Core 0 wants to write X: | v Core 0 requests ownership | +---->  
Core 1's X becomes I | v Core 0: X = M If Core 1 later reads X: | v Core 0 supplies/backs the newer  
value | v Both cores can hold X as S again.

When a core writes to a shared line, it must obtain exclusive ownership and invalidate other cached copies. This is how coherence preserves a consistent value for a memory address. ■filecite■turn0file0■L125-L131■

**Coherence ≠ memory ordering.** MESI can ensure that cores do not permanently disagree about the value of an address, but it does not by itself define the order in which different memory operations become visible. Memory-ordering rules and barriers handle that problem. ■filecite■turn0file0■L135-L136■

## 5. False Sharing: Why Separate Variables Can Still Fight

A cache does not normally move one integer at a time. It moves a **cache line**, commonly 64 bytes. This creates an important performance problem.

Cache line (64 bytes) +-----+ | counter A | counter B |  
other data ... | +-----+ ^ ^ Core 0 Core 1 Core 0  
increments A ↓ Line becomes owned by Core 0 ↓ Core 1's copy is invalidated Core 1 increments B ↓  
Line becomes owned by Core 1 ↓ Core 0's copy is invalidated ↓ PING-PONG

The variables A and B are logically independent, but physically they share a cache line. Every write to one variable can therefore cause coherence traffic affecting the other core. The research document calls this **false sharing** and describes the resulting cache-line ping-pong as a major throughput problem. ■filecite■turn0file0■L140-L155■

### 5.1 The Solution: Cache-Line Padding

```
struct alignas(64) PaddedCounter { std::atomic<long> counter; }; PaddedCounter A; PaddedCounter B;
```

Padding places frequently modified variables on separate cache lines. Core 0 can then update A without repeatedly invalidating the line containing B, and vice versa. The research specifically identifies `alignas(64)` and cache-line padding as a practical mitigation. ■filecite■turn0file0■L164-L165■

**Important insight:** false sharing is a *performance* problem, not necessarily a correctness problem. The coherence protocol still maintains correctness, but the extra ownership transfers can make the program dramatically slower.

## 6. CAS: The Foundation of Lock-Free Algorithms

CAS means **Compare-And-Swap**. It is an atomic operation that says: “Change this value only if it is still equal to the value I expected.”

```
CAS(address, expected, desired) if (*address == expected) { *address = desired; return SUCCESS; }  
else { return FAILURE; }
```

Example: two threads read counter = 10 and both attempt CAS(counter, 10, 11). Only one can succeed. After the first succeeds, the value is 11, so the second CAS fails and can retry with the new value.

■filecite■turn0file0■L214-L240■

Unlike a mutex-based critical section, a failed CAS does not inherently put the thread to sleep or require a kernel context switch. The thread can remain in user space and retry. This can reduce latency and scheduling overhead under suitable workloads, although excessive contention can cause wasted CPU cycles.

■filecite■turn0file0■L241-L242■

## 7. LMAX Disruptor / Ring Buffer

The LMAX Disruptor is a well-known high-performance concurrency design based on a preallocated ring buffer and sequence numbers. Its important ideas are not simply “avoid locks”; they are **avoid unnecessary contention, control memory ordering, and make the data layout cache-friendly**. ■filecite■turn0file0■L256-L260■

```
PREALLOCATED RING BUFFER +-----+-----+-----+-----+-----+-----+ | E0 | E1 | E2 | E3 | E4 | ... |
+-----+-----+-----+-----+-----+ ^ ^ Consumer Producer | | sequence sequence | | +-----
coordination ---+ Producer: 1. Claim a sequence/slot. 2. Write event into the slot. 3. Publish the
sequence with release ordering. Consumer: 1. Acquire-load the published sequence. 2. Read the event.
3. Advance its sequence.
```

The ring buffer is preallocated, so the producer and consumer do not need to allocate a new object for every event. Sequence counters coordinate progress, while cache-line padding reduces false sharing.

■filecite■turn0file0■L256-L276■

### 7.1 Why This Can Avoid Kernel Context Switches

A conventional mutex may block a thread when the lock is unavailable. Blocking can involve OS scheduling and potentially a context switch. A lock-free CAS-based algorithm instead attempts the operation atomically; if another thread wins the race, the current thread can retry.

This does **not** mean the operating system never schedules the thread. The thread can still be preempted. The important point is that the **algorithm itself does not need to sleep in the kernel merely because another thread currently owns a lock**. ■filecite■turn0file0■L284-L287■

**Trade-off:** spinning consumes CPU. Lock-free designs are most attractive when operations are short and contention is manageable, or when predictable low latency is more important than minimizing CPU usage.

## 8. Putting Everything Together

The following chain connects the entire assignment from software source code down to CPU hardware:

```
APPLICATION | | shared data + atomics v MEMORY MODEL | | acquire / release / seq_cst v COMPILER | |
must respect synchronization constraints v CPU CORE | | out-of-order execution | store buffers |
speculative loads v CACHE HIERARCHY | | cache lines v COHERENCE PROTOCOL | | MESI / MOESI |
ownership + invalidation v INTER-CORE TRAFFIC | | cache-line transfers v PERFORMANCE Correctness
question: "Can another thread legally observe this state?" Performance question: "How much
cache-coherence traffic does this design create?"
```

**Core lesson:** memory ordering solves correctness; cache-aware data layout helps solve scalability. They are related, but they are not the same problem.

## 9. Practical Engineering Guidelines

- **Use atomics for shared synchronization state.** Do not rely on ordinary variables when multiple threads access them without a proper synchronization mechanism.
- **Prefer the weakest memory order that correctly expresses the algorithm.** Acquire/release is often sufficient for publication patterns; sequential consistency is simpler but stronger.

- **Separate hot writable variables.** Use cache-line alignment/padding where independent counters or sequence variables are being modified by different cores.
- **Measure rather than assume.** Tools such as Linux perf c2c and Intel VTune can help identify cache-line contention and coherence traffic; ThreadSanitizer can help identify synchronization/data-race issues. [filecite\turn0file0\L290-L315](#)
- **Be aware of ABA in more complex lock-free structures.** A value can change  $A \rightarrow B \rightarrow A$  between a read and a CAS, causing a naïve CAS to miss the fact that something changed. Version tags or safe memory-reclamation techniques can mitigate this. [filecite\turn0file0\L245-L253](#)

## 10. Team Research Insights

Our research led to several important conclusions about modern concurrent systems:

- **Concurrency is not only a software problem.** Correctness depends on the interaction between the programming-language memory model, compiler optimizations, CPU execution, and cache coherence.
- **Visibility and ordering are different concepts.** A cache-coherence protocol can maintain a coherent value while the memory model still requires explicit ordering between operations.
- **Performance can fail even when correctness is perfect.** False sharing can make independent variables interfere because CPUs move cache lines rather than individual variables.
- **Lock-free does not mean free of synchronization.** Lock-free algorithms still use atomic hardware primitives, memory ordering, and coherence traffic; they simply avoid traditional blocking locks.
- **Mechanical sympathy matters.** Understanding cache lines, store buffers, memory ordering, and atomic instructions allows developers to design systems that cooperate with the hardware instead of fighting it.

The uploaded research reaches the same overall conclusion: efficient concurrent systems require considering memory ordering, cache behavior, atomic primitives, alignment, and profiling together.

[filecite\turn0file0\L641-L657](#)

## 11. Final Takeaway

A useful way to remember the entire topic is with three questions:

### 1. Can the other thread see my data correctly?

Use the memory model and synchronization—such as release/acquire—to establish the required happens-before relationship.

### 2. Can both cores safely maintain their cached copies?

Cache-coherence protocols such as MESI/MOESI manage ownership, sharing, and invalidation at cache-line granularity.

### 3. Can the hardware do this efficiently?

Avoid unnecessary cache-line bouncing, false sharing, blocking, and allocation. Use cache-aware layout and carefully designed atomic/lock-free algorithms.

The central lesson is therefore: **correctness comes from explicit memory ordering; scalability comes from minimizing unnecessary contention and coherence traffic.** High-performance systems such as ring buffers and the LMAX Disruptor combine both ideas rather than treating concurrency as merely a matter of adding threads or locks.

## References / Source Basis

Primary source: **Deep Research: Concurrency on Modern Multi-Core CPUs**, the document supplied for this assignment. It references Intel/AMD architecture documentation, the C++11 memory model, Lamport's sequential consistency work, Harvard CS161 notes, and practical concurrency explainers. ■filecite■turn0file0■L661-L665■

**Note:** This document is a structured, simplified rewrite of the supplied research for an EPAM team submission. It preserves the source's core terminology and technical framing while adding explanatory structure and diagrams for readability.